

Lecture Scribe
CSE400 – Fundamentals of Probability in
Computing
Lecture 10: Randomized Min-Cut Algorithm

March 20, 2026

1 1. Lecture Overview

This lecture introduces the minimum cut problem in graphs and studies both deterministic and randomized algorithms used to solve it. The topics covered in the lecture include:

The Min-Cut Problem

Why the Min-Cut algorithm is useful

Definition of Min-Cut

Successful Min-Cut Run

Unsuccessful Min-Cut Run

Max-Flow Min-Cut Theorem

Deterministic Min-Cut Algorithm

Stoer–Wagner Minimum Cut Algorithm

Pseudocode for deterministic min-cut

Randomized Min-Cut Algorithm

Karger’s Randomized Algorithm

Pseudocode of the randomized algorithm

Comparison of deterministic and randomized min-cut algorithms

Theorem describing the probability of finding a min-cut

Python simulation instructions

The lecture aims to show how graph partitioning and network reliability problems can be solved using algorithmic techniques.

2 2. The Min-Cut Problem

2.1 2.1 Why Use Min-Cut?

The min-cut algorithm is applied in problems related to network connectivity, reliability, and optimization.

The lecture describes several application areas.

2.1.1 2.1.1 Network Design

Min-cut helps improve communication efficiency and optimize network flow.

Step-by-step reasoning:

A network can be represented using a graph.

Nodes represent devices or entities in the network.

Edges represent communication links.

Removing certain links may break the network connectivity.

The goal is to determine the minimum capacity cut, which is the smallest set of edges whose removal disconnects the network.

By identifying this set, engineers can analyze weaknesses and optimize the network design.

2.1.2 2.1.2 Communication Networks

Min-cut can help understand network vulnerability.

Reasoning process:

Communication networks must remain functional even when some components fail.

If a small number of edges can disconnect the network, the network is vulnerable.

By computing the minimum cut, these critical edges can be identified.

This helps design robust and fault-tolerant communication systems.

2.1.3 2.1.3 VLSI Design

In Very Large Scale Integration (VLSI) systems, circuits contain many interconnected components.

Reasoning:

Circuit components can be modeled as vertices.

Interconnections between components are represented as edges.

Large circuits often need to be partitioned into smaller components.

Partitioning should minimize the number of connections between partitions.

The min-cut algorithm helps perform such partitioning efficiently and reduces interconnect complexity.

3 3. Definition of Min-Cut

3.1 3.1 Cut-Set

A cut-set in a graph is defined as:

A set of edges whose removal breaks the graph into two or more connected components.

Step-by-Step Interpretation

A graph consists of vertices connected by edges.

The edges maintain connectivity between vertices.
If certain edges are removed, connectivity may be disrupted.
When removing a set of edges divides the graph into separate components, those edges form a cut-set.
Thus, a cut-set represents edges responsible for maintaining connectivity between different parts of the graph.

3.2 Minimum Cut

Given a graph $G = (V, E)$
where
 V is the set of vertices
 E is the set of edges
 n is the number of vertices
The minimum cut problem is defined as:
Finding a cut-set with the smallest possible number of edges in graph G .
Logical Reasoning
A graph may contain several different cut-sets.
Each cut-set contains a specific number of edges.
The cut-set with the smallest number of edges is called the minimum cut.

4. Edge Contraction

The lecture identifies edge contraction as the main operation used in the min-cut algorithm.

4.1 Definition

Edge contraction is an operation that:
Removes an edge from a graph
Merges the two vertices connected by that edge into a single vertex
This process is illustrated in the diagram shown on page 15 of the slides, where two vertices connected by an edge are combined into one vertex after contraction.

4.2 Detailed Edge Contraction Procedure

When contracting an edge (u, v) :
Select the edge connecting vertices u and v .
Merge vertices u and v into a single vertex.
Remove the edge connecting them.
Retain all other edges in the graph.
Two additional properties are mentioned:
The resulting graph may contain parallel edges.
Self-loops are removed from the graph.
Thus, the contraction operation simplifies the graph while maintaining the essential connectivity relationships.

5 5. Successful Min-Cut Run

A successful min-cut run refers to a case where the algorithm successfully finds the minimum cut.

Step-by-Step Reasoning

The algorithm repeatedly performs edge contractions.

Some edges belong to the true minimum cut.

If these edges are not contracted, the structure of the minimum cut remains intact.

The graph eventually reduces to two vertices.

The edges connecting these vertices represent the minimum cut.

The example shown on page 19 of the slides illustrates a sequence of contractions that leads to a correct minimum cut.

6 6. Unsuccessful Min-Cut Run

An unsuccessful run occurs when the algorithm fails to find the correct minimum cut.

Logical Reasoning

The algorithm selects edges randomly for contraction.

If an edge belonging to the true minimum cut is contracted early, the structure of the minimum cut is destroyed.

The final cut produced by the algorithm will therefore not be the true minimum cut.

The example on page 22 of the slides illustrates this situation.

7 7. Max-Flow Min-Cut Theorem

The Max-Flow Min-Cut Theorem establishes a relationship between network flow and graph cuts.

7.1 Theorem Statement

In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

7.2 Definitions Used in the Theorem

7.2.1 Capacity of a Cut

The capacity of a cut is defined as:

The sum of the capacities of edges oriented from a vertex in set X to a vertex in set Y .

7.2.2 Minimum Cut

The cut that has the smallest possible capacity.

7.2.3 Minimum Cut Capacity

The capacity of the minimum cut.

7.2.4 Maximum Flow

The largest amount of flow that can travel from:

Source S

to Sink T

in the network.

The network diagram shown on page 26 illustrates this relationship between maximum flow and minimum cut.

8 8. Deterministic Min-Cut Algorithm

A deterministic algorithm guarantees the exact minimum cut.

The lecture introduces the Stoer–Wagner Minimum Cut Algorithm.

8.1 Stoer–Wagner Theorem

Let

s and t be two vertices in graph G

$G/s, t$ represent the graph obtained by merging vertices s and t

The theorem states:

A minimum cut of G can be obtained by taking the smaller value between:

A minimum s - t cut of G

A minimum cut of the graph $G/s, t$

Logical Explanation

Two possibilities exist:

Case 1

There exists a minimum cut that separates vertices s and t .

In this case, the minimum s - t cut is also the minimum cut of the graph.

Case 2

There is no minimum cut that separates s and t .

In this situation, merging s and t does not affect the minimum cut, so the minimum cut of $G/s, t$ becomes the solution.

9 9. Deterministic Min-Cut Algorithm Pseudocode

9.1 Algorithm 1: MinimumCutPhase(G, a)

Initialize the set $A = \{a\}$

While $A \neq V$

add the vertex that is most tightly connected to set A .

Once all vertices are included in A , return the cut weight, called the cut of the phase.

9.2 Algorithm 2: MinimumCut(G)

While the graph contains vertices:

Choose any vertex a .

Execute the procedure MinimumCutPhase(G, a)

If the cut obtained in the phase is smaller than the current minimum cut, update the minimum cut.

Shrink the graph by merging the last two vertices added.

Continue until the algorithm terminates.

Return the minimum cut.

10 10. Randomized Min-Cut Algorithm

Randomized algorithms incorporate random decisions during execution.

10.1 Why Randomized Algorithms Are Used

The lecture lists several reasons:

Probabilistic guarantee of success

Efficiency

Parallelization

Approximation guarantees

Avoidance of worst-case instances

Robustness

These properties make randomized algorithms useful when deterministic algorithms become computationally expensive.

11 11. Karger's Randomized Algorithm

Karger's algorithm finds the minimum cut through random edge contractions.

11.1 Algorithm Steps

Start with graph G .

Randomly choose an edge.

Contract the selected edge.

Repeat the process until only two vertices remain.

The edges connecting the two vertices form the resulting cut.

The diagram shown on page 34 of the slides illustrates an example run of Karger's algorithm.

12 12. Recursive Randomized Min-Cut Pseudocode

12.1 Algorithm: Recursive-Randomized-Min-Cut(G, α)

Input:

Undirected multigraph G with n vertices

Integer constant $\alpha > 0$

Output:

A cut C

Algorithm Steps

If $n \leq \alpha^3$

then compute the minimum cut using brute force search.

Otherwise repeat for $i = 1$ to α :

a. Generate graph G' by performing random contraction steps.

b. Compute recursively $C' = \text{RecursiveRandomizedMinCut}(G', \alpha)$

c. If $i = 1$ or $|C'| < |C|$

update $C = C'$

Return C .

13 13. Comparison: Deterministic vs Randomized Min-Cut

13.1 Deterministic Min-Cut

Characteristics:

Always guarantees an exact minimum cut.

May have higher time complexity for large graphs.

Time complexity:

$O(VE + V^2 \log V)$

13.2 Randomized Min-Cut

Characteristics:

Produces an approximate minimum cut with high probability.

Faster for large graphs.

Time complexity:

$O(V^2)$

Thus, the choice of algorithm depends on the specific problem requirements.

14 14. Probability Theorem for Min-Cut

The lecture provides the probability guarantee for the randomized algorithm.

The probability that the algorithm outputs a minimum cut is at least $\frac{2}{n(n-1)}$ where n is the number of vertices.

Because this probability can be small, the algorithm is typically executed multiple times to increase the likelihood of finding the correct minimum cut.

15 15. Python Simulation

Students are instructed to:

Open the Campuswire post for Lecture 10.

Download the Jupyter Notebook (.ipynb) file provided in the comments.

Execute the notebook to observe the randomized min-cut algorithm in practice.

16 Final Summary

This lecture introduces the minimum cut problem and the algorithms used to solve it.

The main ideas developed throughout the lecture include:

Definition of cut-sets and minimum cuts

Edge contraction as the key operation in min-cut algorithms

Successful and unsuccessful runs of randomized algorithms

The Max-Flow Min-Cut theorem

The Stoer–Wagner deterministic algorithm

Karger’s randomized min-cut algorithm

Complexity analysis and probability guarantees

Together, these concepts illustrate how graph algorithms and probabilistic techniques can be applied to analyze and optimize networks.